

Snowflake Stages & Data Loading -- Real-Time Project Scenario

1 What is a Stage in Snowflake?

A **stage** is a **temporary storage location** used to **store files before loading them into Snowflake tables** or **after unloading data**.

👉 Stages help Snowflake access files from:

- Local machines
- Cloud storage (AWS S3, Azure Blob, GCS)
- Snowflake-managed storage

2 Types of Stages in Snowflake

◆ 1. Internal Stages (Snowflake-Managed)

Files are stored **inside Snowflake**.

Types:

1. **Named Internal Stage** (Manually created)
2. **Table Stage** (Auto-created for every table)
3. **User Stage** (Auto-created for every user)

◆ 2. External Stages

Files are stored **outside Snowflake**:

- AWS S3
- Azure Blob
- Google Cloud Storage

3 Internal Stage

What is it?

A stage you **explicitly create** and manage.
Used heavily in **real projects**.

Create Named Internal Stage

CREATE OR REPLACE STAGE internal_stg

DIRECTORY = (ENABLE = TRUE);

Why DIRECTORY = ENABLE = TRUE?

- Enables **directory table**
- Allows browsing files like folders
- Required for LIST command

✦ Access Syntax

@internal_stg

🔗 External Stage (Quick Overview)

CREATE OR REPLACE STAGE ext_stg

URL = 's3://mybucket/data/'

CREDENTIALS = (AWS_KEY_ID='xxx' AWS_SECRET_KEY='yyy');

⚠ Not recommended in real projects

- ✓ Use **Storage Integration** instead (secure & best practice)

5 Table Stage

✓ What is it?

- Automatically created **for every table**
- Table-specific

✦ Access Syntax

@%customers

⚠ Files in a table stage **can be loaded only into that table**

6 User Stage

✓ What is it?

- Automatically created for every user
- User-specific

✦ Access Syntax

@~

or

@~username

7 Uploading Files – IMPORTANT RULE 🚨

❌ PUT command **CANNOT** be run in Snowflake Worksheet

✅ Must use **SnowSQL CLI**

8 SnowSQL (Command Line Interface)

♦ Why SnowSQL?

Used when:

- Files are on **local machine**
- Automation is required
- Scheduling jobs

♦ Install SnowSQL

- Download from Snowflake website
- Install on **machine/server where files exist**

Verify installation:

```
snowsql -v
```

9 Connect to Snowflake Using SnowSQL

```
snowsql -a <account_name> -u <username>
```

Example:

```
snowsql -a xy12345.ap-south-1.aws -u skill_scalar
```

👉 Enter password when prompted

10 Upload File to Internal Stage (PUT Command)

♦ Syntax

```
PUT file://C:/data/customers.csv @internal_stg/DW/customer/;
```

♦ What happens internally?

- File is **automatically compressed**
- customers.csv → customers.csv.gz

✓ Compression reduces storage & improves performance

1 1 Verify Uploaded Files

LIST @internal_stg/DW/customer;

1 2 Create Target Table

CREATE OR REPLACE TABLE customers (

id INT,

name STRING,

city STRING,

salary NUMBER

);

1 3 Load Data into Table (COPY Command)

◆ Basic COPY Command

COPY INTO customers

FROM @internal_stg/DW/customer

FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1);

1 4 Handling Data Errors (Comma Inside Data)

✗ Problem

Comma inside a field causes column mismatch.

✓ Solution

FILE_FORMAT = (

TYPE = CSV

SKIP_HEADER = 1

FIELD_OPTIONALLY_ENCLOSED_BY = ''''

);

1 5 Reloading Same File (FORCE Option)

✗ Default Behavior

Already-loaded file → **0 files processed**

✅ Use FORCE

COPY INTO customers

FROM @internal_stg/DW/customer

FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1)

FORCE = TRUE;

📊 Row count increases again

1 6 Automatically Remove File After Load (PURGE)

COPY INTO customers

FROM @internal_stg/DW/customer

FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1)

FORCE = TRUE

PURGE = TRUE;

✔ File removed from stage after successful load

1 7 PUT vs COPY (Very Important Interview Question)

Command	Purpose
PUT	Upload file from local machine to internal stage
COPY	Load data from stage into Snowflake table

👉 PUT ≠ Load

👉 COPY = Load

1 8 Internal vs External Load Flow

◆ Internal Stage (2 Steps)

1. PUT → Upload file
2. COPY → Load data

◆ External Stage (1 Step)

```
COPY INTO table_name  
FROM @external_stage;
```

1 9 GET Command (Download Files)

```
GET @internal_stg/DW/customer file://C:/output/;
```

Used for **data unloading** (covered later)

2 0 Real-Time vs Automation

Scenario	Tool
Manual load	Snowflake Worksheet
Local/Server files	SnowSQL
Streaming	Snowpipe
Scheduled jobs	SnowSQL + scheduler

Interview-Ready Summary

- Stages store files before loading
- Internal stages store data inside Snowflake
- External stages refer to cloud storage
- PUT → Upload (SnowSQL only)
- COPY → Load
- FORCE reloads files
- PURGE removes files after load
- Table & user stages are auto-created
- Named stages are used in real projects

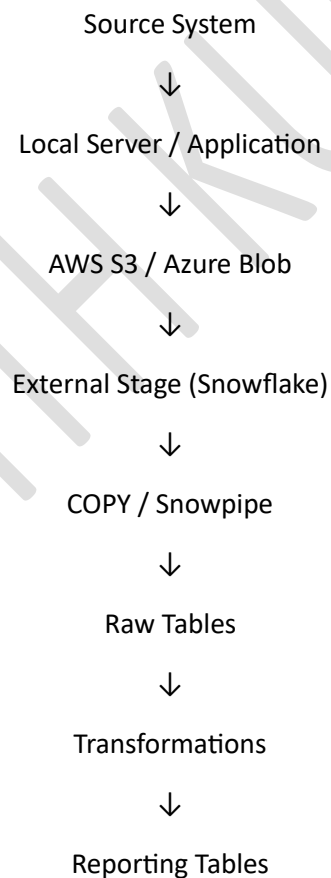
Real-Time Project Scenario: Snowflake Data Ingestion

Business Requirement

A **Banking / Retail / Insurance** company receives **daily customer transaction files** from multiple source systems.

- Files arrive **every day**
 - Size: MBs to few GBs
 - Format: CSV
 - Source: **Local server → Cloud storage**
 - Target: **Snowflake Data Warehouse**
 - SLA: Data must be available by **7 AM**
-

High-Level Architecture



Scenario 1: Batch Load Using External Stage (Most Common)

◆ Why External Stage?

- Files already in **cloud storage**
- No need to upload via SnowSQL
- Faster & scalable

◆ Step 1: Create Storage Integration (Best Practice)

```
CREATE OR REPLACE STORAGE INTEGRATION s3_int
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = S3
ENABLED = TRUE
STORAGE_ALLOWED_LOCATIONS = ('s3://bank-transactions/');
```

- ✓ Secure
- ✓ No hardcoded keys

◆ Step 2: Create External Stage

```
CREATE OR REPLACE STAGE txn_ext_stage
URL = 's3://bank-transactions/daily/'
STORAGE_INTEGRATION = s3_int
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1);
```

◆ Step 3: Create Raw Table

```
CREATE OR REPLACE TABLE raw_transactions (
  txn_id STRING,
  account_id STRING,
  amount NUMBER,
  txn_date DATE,
  load_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```


◆ Step 4: Load Data (COPY Command)

COPY INTO raw_transactions

FROM @txn_ext_stage

ON_ERROR = 'CONTINUE';

💡 In real projects:

- Errors are logged
- Bad records handled separately

◆ Step 5: Prevent Duplicate Loads

Snowflake tracks loaded files automatically

If the same file is loaded again:

👉 **0 files processed**

Unless:

FORCE = TRUE

Scenario 2: Local Server → Snowflake (Internal Stage + SnowSQL)

◆ When This is Used?

- Legacy systems
- Vendor sends files via SFTP
- Files land on **on-prem server**

◆ Step 1: Create Internal Stage

```
CREATE OR REPLACE STAGE int_stage  
DIRECTORY = (ENABLE = TRUE);
```

◆ Step 2: Upload File Using SnowSQL

```
PUT file://C:/data/transactions.csv @int_stage/daily/;
```

- ✓ File auto-compressed
- ✓ Folder auto-created

◆ Step 3: Load Data

```
COPY INTO raw_transactions  
FROM @int_stage/daily  
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1)  
PURGE = TRUE;
```

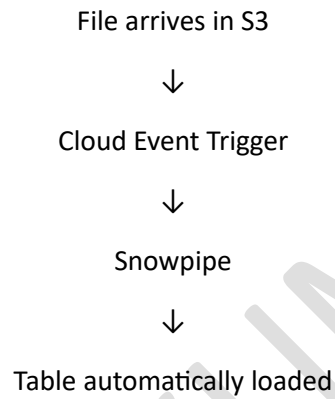
- ✓ File removed after load

Scenario 3: Near Real-Time Data Using Snowpipe

◆ Business Need

- Data every **5–10 minutes**
- No manual COPY execution

◆ How Snowpipe Works



◆ Create Snowpipe

```
CREATE OR REPLACE PIPE txn_pipe
AUTO_INGEST = TRUE
AS
COPY INTO raw_transactions
FROM @txn_ext_stage;
```

- ✓ Serverless
- ✓ Event-driven
- ✓ No scheduler needed

Scenario 4: Transform & Publish Data

◆ Step 1: Clean & Transform

```
CREATE OR REPLACE TABLE curated_transactions AS  
  
SELECT  
  
    txn_id,  
  
    account_id,  
  
    amount,  
  
    txn_date  
  
FROM raw_transactions  
  
WHERE amount > 0;
```

◆ Step 2: Schedule Using TASK

```
CREATE OR REPLACE TASK daily_txn_task  
  
WAREHOUSE = compute_wh  
  
SCHEDULE = 'USING CRON 0 6 * * * UTC'  
  
AS  
  
INSERT INTO curated_transactions  
SELECT * FROM raw_transactions;
```

Error Handling in Real Projects

✓ Validation Before Load

```
COPY INTO raw_transactions  
FROM @txn_ext_stage  
VALIDATION_MODE = RETURN_ERRORS;
```

✓ Bad Data Table

```
ON_ERROR = 'CONTINUE';
```

Bad rows are logged and analyzed later.

How to Explain This in Interview (Best Answer)

"In my project, we receive daily transaction files in AWS S3. We created a storage integration and an external stage to securely access the files. Data is loaded into raw tables using COPY commands, and for near real-time ingestion we use Snowpipe. We handle errors using validation mode and ON_ERROR options. Transformations are done using SQL and scheduled using Snowflake Tasks."